

Gen-ai ~

Assignment - 3

Multi-Agent Workflow Design

Name : Mohammed Afnan

Ph. No. : 8790112677

Email : afnanmohd5578@gmail.com

1. Introduction to Multi-Agent Systems

A Multi-Agent System (MAS) is an AI architecture where multiple autonomous agents collaborate to accomplish complex tasks that a single agent cannot efficiently handle alone. Each agent has a defined role, a set of tools, and a communication protocol to interact with other agents in the system.

Frameworks such as [CrewAI](#), [LangChain](#), and [Semantic Kernel](#) provide ready-to-use abstractions for building these systems, enabling developers to define agent roles, assign tools, manage memory, and orchestrate communication flows with minimal boilerplate.

Framework Comparison

Framework	Primary Use	Key Feature	Language Support
CrewAI	Role-based agent teams	Crew, Agent, Task abstractions	Python
LangChain	LLM chains & agents	Tools, Memory, Chains, LCEL	Python, JS
Semantic Kernel	Enterprise AI orchestration	Plugins, Planners, Skills	Python, .NET, Java

2. Agent Roles and Responsibilities

In a Multi-Agent Team, each agent is assigned a specific role with a defined goal and access to relevant tools. Below are the four key agents in this workflow, implemented using CrewAI, LangChain, and Semantic Kernel conventions.

Orchestrator Agent (Manager)	
Role	Plans the overall task, breaks it into sub-tasks, and delegates to the right agents.
Goal	Ensure the final output meets user requirements by coordinating all agents.
Tools	None — uses reasoning and planning only
Frameworks	CrewAI: Process.hierarchical LangChain: AgentExecutor with router Semantic Kernel: Planner
Example	User asks 'Write a blog post about AI.' Orchestrator delegates: research > write > review.
Code	<pre>Agent(role='Manager', goal='Coordinate tasks', allow_delegation=True)</pre>

Research Agent	
Role	Searches the web and documents to gather relevant, up-to-date information.

Goal	Provide accurate context and summaries to the Writer Agent.
Tools	Web Search (SerperDev), PDF Reader, Wikipedia Tool
Frameworks	CrewAI: Tool-equipped Agent LangChain: ReAct Agent Semantic Kernel: Plugin with search skill
Example	Searches 'AI trends 2024' and returns a structured summary with sources.
Code	<code>Agent(role='Researcher', tools=[search_tool, pdf_tool], verbose=True)</code>

Writer Agent	
Role	Takes research output and generates a well-structured written response.
Goal	Produce clear, engaging, and accurate content based on the research provided.
Tools	Text Generator, Markdown Formatter, Template Tool
Frameworks	CrewAI: Task-based Agent LangChain: LLMChain with prompt template Semantic Kernel: Skill function
Example	Receives research summary and writes a 500-word blog post in markdown format.
Code	<code>Agent(role='Writer', goal='Write high quality content', verbose=True)</code>

Critic Agent (Reviewer)	
Role	Reviews the Writer Agent's output for quality, accuracy, and completeness.
Goal	Return feedback if quality is low; approve if the output meets the criteria.
Tools	Fact Checker, Grammar Scorer, Similarity Tool
Frameworks	CrewAI: Review Task LangChain: Evaluation chain Semantic Kernel: Validator plugin
Example	Reads the draft, checks facts, scores it 8/10, and approves for final delivery.
Code	<code>Task(description='Review and score the content', agent=critic_agent)</code>

3. Communication Flow

The communication flow defines how agents pass information between each other. In CrewAI, this is managed via Tasks and the Process type. In LangChain, agents communicate through chain outputs passed as inputs to the next chain. In Semantic Kernel, the Planner orchestrates function calls across plugins.

Step	Phase	Agent	Action
------	-------	-------	--------

1	User Input	User	Sends query to the Orchestrator Agent
2	Planning	Orchestrator	Breaks query into sub-tasks, assigns to agents
3	Research	Research Agent	Uses tools to gather and summarize context
4	Writing	Writer Agent	Generates structured content using research
5	Review	Critic Agent	Scores output; sends feedback if score is low
6	Feedback Loop	Writer Agent	Revises content based on Critic feedback
7	Final Output	Orchestrator	Returns approved output to the user

4. CrewAI Implementation

CrewAI is a framework for orchestrating role-playing AI agents. It uses a Crew to group agents and Tasks to define what each agent should do. The Process can be sequential or hierarchical.

```

from crewai import Agent, Task, Crew, Process

from crewai_tools import SerperDevTool

search_tool = SerperDevTool()

# Define Agents

researcher = Agent(

    role='Research Specialist',

    goal='Find accurate information on the given topic',

    tools=[search_tool],

    verbose=True

)

```

```
writer = Agent(  
  
    role='Content Writer',  
  
    goal='Write a well-structured blog post using the research',  
  
    verbose=True  
  
)
```

```
critic = Agent(  
  
    role='Content Reviewer',  
  
    goal='Review and score the blog post for quality',  
  
    verbose=True  
  
)
```

```
# Define Tasks
```

```
research_task = Task(  
  
    description='Research the topic: {topic}',  
  
    agent=researcher,  
  
    expected_output='A structured research summary'  
  
)
```

```
write_task = Task(  
  
    description='Write a 500-word blog post using the research',  
  
    agent=writer,  
  
    expected_output='A complete blog post in markdown'  
  
)
```

```
review_task = Task(  
  
    description='Review the blog post for quality and accuracy',  
  
    agent=critic,
```

```

        expected_output='Approved post or feedback with score'
    )

# Create and run the Crew

crew = Crew(

    agents=[researcher, writer, critic],

    tasks=[research_task, write_task, review_task],

    process=Process.sequential

)

result = crew.kickoff(inputs={'topic': 'AI trends in 2024'})

print(result)

```

5. LangChain Implementation

LangChain allows building multi-agent systems using chains, tools, and agents. The LCEL (LangChain Expression Language) makes it easy to pipe outputs between agents. It supports memory, tools, and custom prompts.

```

from langchain_groq import ChatGroq

from langchain.agents import AgentExecutor, create_react_agent

from langchain_community.tools import DuckDuckGoSearchRun

from langchain.prompts import PromptTemplate

from langchain_core.output_parsers import StrOutputParser

llm = ChatGroq(model='llama-3.3-70b-versatile', api_key='YOUR_API_KEY')

search = DuckDuckGoSearchRun()

```

```

# Research Agent using ReAct pattern

research_prompt = PromptTemplate.from_template(

    'Research this topic and summarize: {input}\n{agent_scratchpad}'

)

research_agent = create_react_agent(llm, [search], research_prompt)

research_executor = AgentExecutor(agent=research_agent, tools=[search])


# Writer Chain using LCEL

write_chain = (

    PromptTemplate.from_template('Write a blog post using: {research}')

    | llm

    | StrOutputParser()

)


# Critic Chain

critic_chain = (

    PromptTemplate.from_template('Review and score this content out of 10: {draft}')

    | llm

    | StrOutputParser()

)


# Full Pipeline

topic = 'AI trends in 2024'

research_output = research_executor.invoke({'input': topic})

blog_post = write_chain.invoke({'research': research_output['output']})

review = critic_chain.invoke({'draft': blog_post})

print(review)

```

6. Semantic Kernel Implementation

Semantic Kernel is Microsoft's open-source SDK for integrating LLMs. It uses Plugins (skills) and a Planner to orchestrate multi-step AI workflows. It natively supports Python, .NET, and Java.

```
import asyncio

import semantic_kernel as sk

from semantic_kernel.connectors.ai.open_ai import OpenAIChatCompletion

kernel = sk.Kernel()

kernel.add_chat_service(
    'chat', OpenAIChatCompletion('gpt-4', api_key='YOUR_KEY')
)

# Define Plugins (Skills)

research_plugin = kernel.create_semantic_function(
    'Search and summarize information about: {{$input}}',
    function_name='Research',
    plugin_name='ResearchPlugin',
    max_tokens=500
)

write_plugin = kernel.create_semantic_function(
    'Write a detailed blog post based on: {{$input}}',
    function_name='Write',
    plugin_name='WriterPlugin',
    max_tokens=800
)
```

```

review_plugin = kernel.create_semantic_function(

    'Review and score this content out of 10: {{$input}}',

    function_name='Review',

    plugin_name='CriticPlugin',

    max_tokens=200

)

# Run the pipeline

async def run_pipeline(topic):

    research = await kernel.run(research_plugin, input_str=topic)

    draft    = await kernel.run(write_plugin,      input_str=str(research))

    review   = await kernel.run(review_plugin,     input_str=str(draft))

    print('Final Review:', review)

asyncio.run(run_pipeline('AI trends in 2024'))

```

7. Real-World Example: AI Blog Post Generator

Task: "Write a blog post about the impact of AI in healthcare."

Agent	Action	Output
User	Sends query to system	"Write a blog about AI in healthcare"
Orchestrator	Plans and delegates tasks	Assigns research, write, review tasks
Research Agent	Searches web and PDFs	Summary: AI in diagnostics, drug discovery...
Writer Agent	Generates blog post	500-word structured post in markdown
Critic Agent	Reviews the post	Score: 9/10 — Approved for delivery

Orchestrator	Returns final output	Sends approved blog post back to user
--------------	----------------------	---------------------------------------

8. Conclusion

Multi-Agent Systems represent a powerful paradigm for solving complex AI tasks by dividing responsibilities among specialized agents. Frameworks like CrewAI, LangChain, and Semantic Kernel make it practical to implement these systems in Python with minimal setup.

- Each agent has a single, well-defined role — this keeps the system modular and maintainable.
- The Orchestrator (Manager) is the brain — it plans, delegates, and consolidates results.
- Communication flow follows sequential or hierarchical process depending on the framework.
- Feedback loops (Critic to Writer) improve output quality before final delivery.
- CrewAI is best for quick role-based teams; LangChain for flexible chains; Semantic Kernel for enterprise.